

PROGRAMAÇÃO ORIENTADA A OBJETOS EM JAVA

Aula 6: Sobrecarga de Métodos e Construtores

Nesta aula, exploraremos o conceito de **sobrecarga (overloading)** — um dos recursos mais poderosos e flexíveis da linguagem Java, que permite a criação de múltiplas versões de um mesmo método ou construtor dentro de uma classe.

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de compreender e aplicar os conceitos fundamentais de sobrecarga em Java com segurança e boas práticas.

1

Compreender Sobrecarga

Entender o conceito de overloading e como ele funciona na linguagem Java.

2

Sobrecarga de Métodos

Aplicar sobrecarga de métodos em diferentes contextos práticos.

3

Sobrecarga de Construtores

Utilizar construtores sobrecarregados para inicializar objetos de forma flexível.

4

Overloading vs. Overriding

Diferenciar sobrecarga de sobreposição e reconhecer quando usar cada uma.

5

Boas Práticas

Reconhecer boas práticas e evitar erros comuns no uso da sobrecarga.

O que é Sobrecarga?

A **sobrecarga (method overloading)** é a capacidade de definir múltiplos métodos com o **mesmo nome** dentro de uma classe, desde que possuam **assinaturas diferentes** — ou seja, diferem no tipo, na quantidade ou na ordem dos parâmetros. Pense em uma ferramenta multifuncional: o mesmo nome de ação, com comportamentos distintos conforme o contexto.

Mesmo Nome

Vários métodos compartilham o mesmo identificador dentro da classe.

Assinatura Diferente

Diferem em tipo, quantidade ou ordem dos parâmetros.

Flexibilidade

Permite reutilização de código e uma API mais intuitiva para o desenvolvedor.



Regras da Sobrecarga em Java

Para que a sobrecarga seja válida em Java, o compilador aplica um conjunto de regras estritas. Compreender essas regras evita erros de compilação e ambiguidades indesejadas no código.

O que DEVE ser diferente

- Lista de parâmetros: tipo, quantidade ou ordem
- A combinação desses elementos forma a assinatura única do método

O que PODE variar livremente

- Tipo de retorno (não define a assinatura)
- Modificadores de acesso (public, private, protected)
- Exceções verificadas (checked exceptions)

❏ **Atenção:** Alterar apenas o tipo de retorno **não** caracteriza sobrecarga e gera erro de compilação!

Assinatura de Método

A **assinatura** de um método é formada pelo seu **nome** mais a **lista de parâmetros** (tipos e ordem). O tipo de retorno *não* faz parte da assinatura — por isso, dois métodos com mesmo nome e mesmos parâmetros, mas tipos de retorno diferentes, causam erro de compilação: `"method already defined"`.

✓ Assinaturas Válidas (Sobrecarga Correta)

`calcular(int a)` — um parâmetro inteiro

`calcular(double a)` — tipo diferente

`calcular(int a, int b)` — quantidade diferente

✗ Assinatura Inválida (Erro de Compilação)

`int calcular(int a)`

`double calcular(int a)` — mesmo nome e parâmetros, só o retorno difere. O compilador **não consegue distinguir** qual chamar!

Sobrecarga de Métodos — Exemplo Prático

A classe `Calculadora` abaixo demonstra três versões do método `somar()`. O compilador identifica automaticamente qual versão invocar com base nos argumentos passados na chamada.

```
public class Calculadora {  
  
    // Versão 1: dois inteiros  
    public int somar(int a, int b) {  
        return a + b;  
    }  
  
    // Versão 2: dois doubles  
    public double somar(double a, double b) {  
        return a + b;  
    }  
  
    // Versão 3: três inteiros  
    public int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

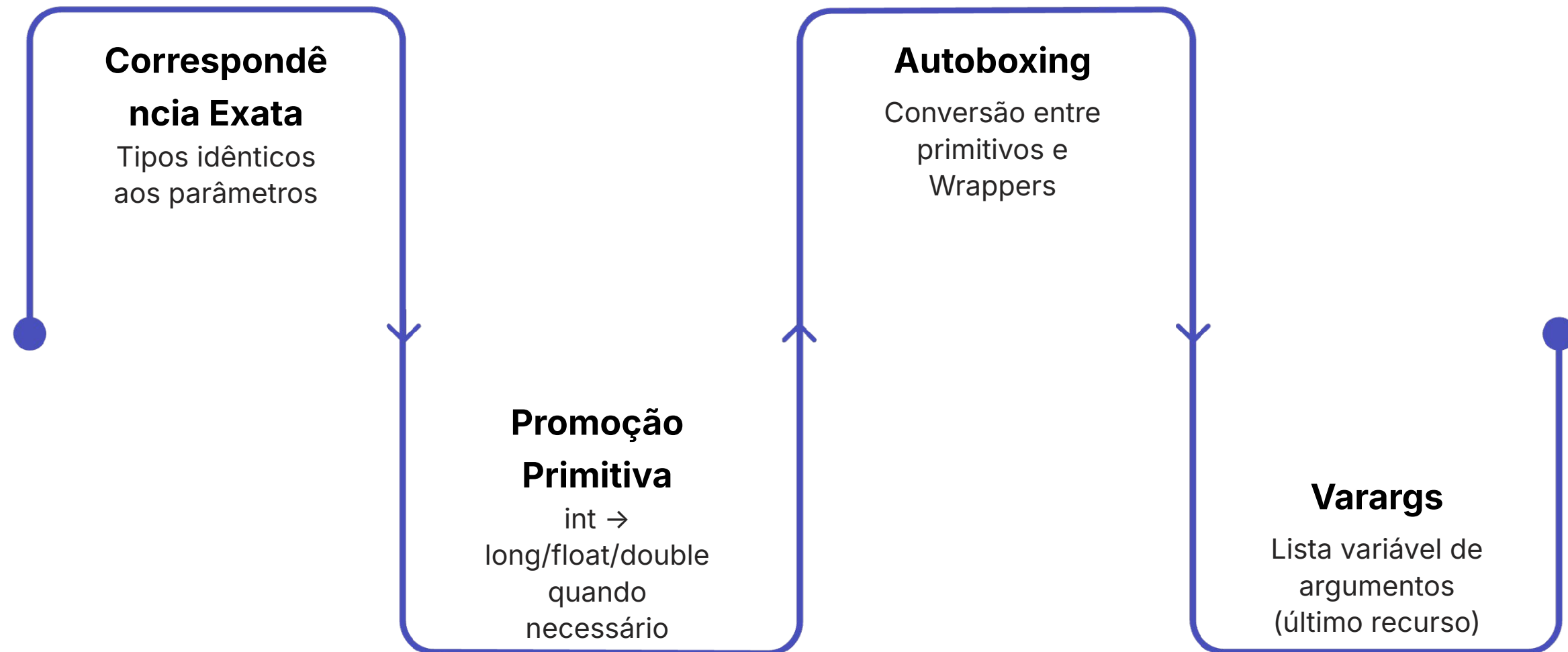
● `somar(2, 3)` → chama a versão `int, int` → retorna `5`

● `somar(2.5, 3.5)` → chama a versão `double, double` → retorna `6.0`

● `somar(1, 2, 3)` → chama a versão com `três inteiros` → retorna `6`

Resolução de Sobrecarga pelo Compilador

Quando um método sobrecarregado é chamado, o compilador Java segue uma **ordem de precedência** para determinar qual versão invocar. Esse processo ocorre em **tempo de compilação** (ligação estática), não em tempo de execução.



O compilador sempre escolhe a opção **mais específica** disponível. Se nenhuma fase resultar em correspondência, ocorre erro de compilação. Evite criar sobrecargas ambíguas que dificultem a resolução.

Sobrecarga de Construtores

Assim como métodos comuns, os **construtores** também podem ser sobrecarregados. Isso permite criar instâncias de uma classe de diferentes formas, oferecendo **flexibilidade** ao desenvolvedor que usa a classe. Cada construtor representa uma maneira distinta de inicializar o objeto.

Construtor Padrão

Sem parâmetros. Inicializa atributos com valores padrão definidos pelo desenvolvedor.

Construtor Parcial

Recebe apenas alguns atributos, deixando os demais com valores padrão.

Construtor Completo

Recebe todos os atributos necessários para uma inicialização completa do objeto.



Exemplo: Sobrecarga de Construtores

A classe `Aluno` abaixo mostra três construtores com assinaturas diferentes, permitindo criar objetos com diferentes níveis de informação disponível no momento da instanciação.

```
public class Aluno {  
    private String nome;  
    private int idade;  
    private String curso;  
  
    // Construtor padrão  
    public Aluno() {  
        this.nome = "Não informado";  
    }  
  
    // Construtor com nome  
    public Aluno(String nome) {  
        this.nome = nome;  
    }  
}
```

```
    // Construtor completo  
    public Aluno(String nome, int idade,  
        String curso) {  
        this.nome = nome;  
        this.idade = idade;  
        this.curso = curso;  
    }  
}
```

❏ **Uso prático:** `new Aluno()`, `new Aluno("Maria")` ou `new Aluno("João", 20, "Ciência da Computação")` — todos válidos!

Chamada entre Construtores com `this()`

O operador `this()` permite que um construtor **chamar outro construtor** da mesma classe, eliminando duplicação de código. Esta técnica é chamada de **encadeamento de construtores** e torna o código mais limpo e fácil de manter.

1 `this()` deve ser a PRIMEIRA instrução

Qualquer outra instrução antes de `this()` gera erro de compilação imediato. Sem exceções.

2 Evita duplicação de lógica

A lógica de inicialização fica centralizada no construtor mais completo, os demais apenas delegam.

3 Encadeamento em cascata

Um construtor pode chamar outro que, por sua vez, chama um terceiro — formando uma cadeia de inicialização.

Exemplo: Encadeamento de Construtores

A classe `Produto` demonstra o padrão de encadeamento: cada construtor delega ao próximo, adicionando valores padrão. Toda a lógica real de atribuição fica no construtor mais completo.

```
public class Produto {  
    private String nome;  
    private double preco;  
    private int quantidade;  
  
    public Produto() {  
        this("Produto padrão");  
    }  
  
    public Produto(String nome) {  
        this(nome, 0.0);  
        // chama Produto(String, double)  
    }  
  
    public Produto(String nome, double preco) {  
        this(nome, preco, 1);  
        // chama o construtor completo  
    }  
}
```

```
public Produto(String nome, double preco, int  
qtd) {  
    this.nome = nome;  
    this.preco = preco;  
    this.quantidade = qtd;  
}  
}
```

Toda inicialização real acontece apenas no último construtor. Os demais apenas definem valores padrão e delegam — padrão elegante e sem repetição!

Sobrecarga vs. Sobreposição

É fundamental distinguir **sobrecarga (overloading)** de **sobreposição (overriding)**. Embora os nomes sejam parecidos, tratam-se de mecanismos completamente diferentes. Esta comparação será retomada nas Aulas 8 (Herança) e 9 (Polimorfismo).

Aspecto	Sobrecarga (Overloading)	Sobreposição (Overriding)
Onde ocorre	Mesma classe	Classes diferentes (herança)
Assinatura	Deve ser diferente	Deve ser idêntica
Tipo de retorno	Pode ser diferente	Mesmo tipo ou subtipo
Ligação	Tempo de compilação	Tempo de execução
Anotação	Não utiliza anotação	Usa @Override
Polimorfismo	Estático (ad-hoc)	Dinâmico (runtime)



Boas Práticas de Sobrecarga

Usar sobrecarga de forma indiscriminada pode tornar o código confuso e difícil de manter. Siga estas diretrizes para garantir clareza e qualidade no seu código.

Semântica Consistente

Use sobrecarga somente quando as versões do método realizam a **mesma operação lógica**. Métodos com mesmo nome devem ter comportamento semanticamente equivalente.

Evite Ambiguidade

Sobrecargas que podem ser chamadas com os mesmos argumentos por promoção de tipo geram confusão. Prefira combinações de parâmetros que sejam claramente distintas.

Documente as Diferenças

Use Javadoc para descrever o comportamento específico de cada versão sobrecarregada, especialmente os valores padrão assumidos.

Encadeie Construtores

Prefira o uso de `this()` para evitar duplicação de lógica entre construtores. A clareza do código deve sempre prevalecer.

Erros Comuns na Sobrecarga

Conhecer os erros mais frequentes ajuda a evitá-los e a depurar problemas mais rapidamente. Veja os principais equívocos cometidos por iniciantes em sobrecarga Java.

✗ Mesmo nome, só retorno diferente

`int calcular(int a)` e `double calcular(int a)` — **erro de compilação!** O compilador não usa o tipo de retorno para distinguir métodos sobrecarregados.

✗ `this()` não é a primeira instrução

Colocar qualquer código antes de `this()` no construtor causa erro de compilação imediato. Mesmo uma simples atribuição antes já invalida o código.

✗ Ambiguidade entre sobrecargas

`metodo(int a, double b)` e `metodo(double a, int b)`: chamar `metodo(1, 2)` gera **ambiguidade** por promoção de tipos — erro de compilação.

✗ Atributos não inicializados

Esquecer de inicializar todos os atributos em cada construtor pode deixar o objeto em estado inconsistente. Use encadeamento com `this()` para evitar isso.

Exercícios de Fixação

Pratique os conceitos aprendidos nesta aula com os exercícios abaixo. Eles cobrem desde a criação de sobrecargas simples até cenários mais desafiadores envolvendo encadeamento de construtores.

- 1** **Classe Retangulo**
Crie construtores sobrecarregados: padrão (quadrado 1×1), apenas largura (quadrado), e largura + altura completo.
- 2** **Classe Conversor**
Implemente métodos `converter()` sobrecarregados: `int→double`, `String→int`, `double→String` com formatação.
- 3** **Caça ao Erro**
Identifique e corrija os erros de sobrecarga em um trecho de código fornecido pelo professor no IDE.
- 4** **Classe Livro**
Crie quatro construtores encadeados com `this()`: padrão, com título, com título+autor, e completo (título, autor, ano, ISBN).
- 5**  **Desafio: Classe Matematica**
Implemente `potencia()`, `raiz()` e `media()` com múltiplas sobrecargas, incluindo versão com `varargs`.